

TC - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continuous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste0(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 3997 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 3953 values"
## [1] "44 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	505	34	120	NA	NA	659
2	63	NA	397	89	106	655
3	275	NA	165	196	15	651
4	288	NA	205	65	33	591
5	206	NA	198	63	36	503
6	190	1	115	67	20	393
7	154	NA	89	24	18	285
8	78	NA	48	20	3	149
9	62	NA	1	NA	4	67

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7663126	0.0515933	0.1820941	NA	NA	659
2	0.0961832	NA	0.6061069	0.1358779	0.1618321	655
3	0.4224270	NA	0.2534562	0.3010753	0.0230415	651
4	0.4873096	NA	0.3468697	0.1099831	0.0558376	591
5	0.4095427	NA	0.3936382	0.1252485	0.0715706	503
6	0.4834606	0.0025445	0.2926209	0.1704835	0.0508906	393
7	0.5403509	NA	0.3122807	0.0842105	0.0631579	285
8	0.5234899	NA	0.3221477	0.1342282	0.0201342	149
9	0.9253731	NA	0.0149254	NA	0.0597015	67

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                        aes(x=pos,y=percent,group=syll_component,
                           color=syll_component,
                           linetype = syll_component,
                           shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=sy
```

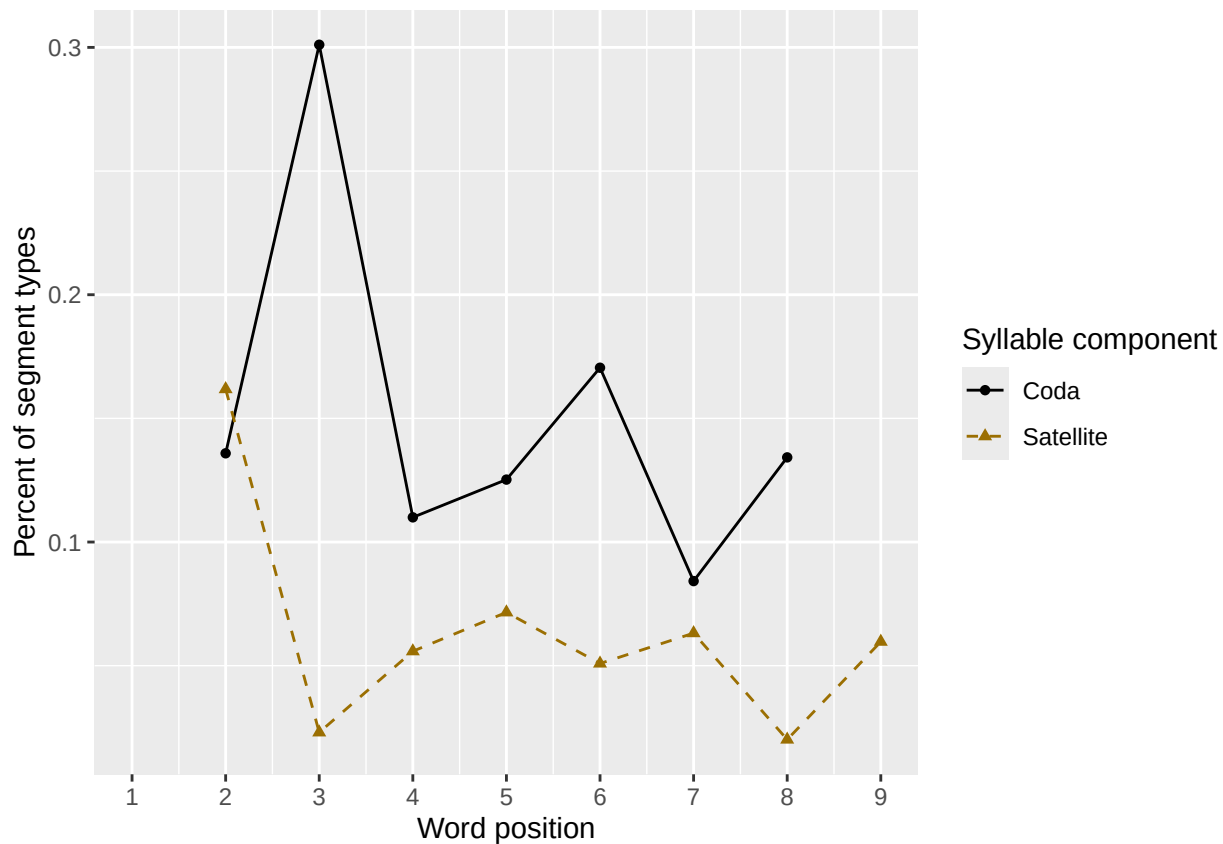
Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_line()``).

Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).

syll_comp_plot

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).



```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 1     1     1    NA    NA    NA    NA    NA    NA
## 2      5 0.978 0.978 0.967 0.944 NA    NA    NA    NA    NA
## 3      6 0.991 0.974 0.991 0.946 0.904 NA    NA    NA    NA
## 4      7 0.991 0.963 0.981 0.981 0.954 0.926 NA    NA    NA
## 5      8 0.993 0.970 0.992 0.947 0.947 0.902 0.837 NA    NA
## 6      9 0.976 0.964 0.988 0.976 0.940 0.917 0.890 0.852 NA
## 7     10 1     0.985 0.970 0.926 0.909 0.838 0.868 0.868 0.776

```

```
# len/pos table
```

```
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                                "_N_by_len_position.csv"))

```

```
pos_len_N_table
```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4    58    58    58    NA    NA    NA    NA    NA    NA
## 2      5    91    89    91    90    NA    NA    NA    NA    NA
## 3      6   115   115   112   111   114    NA    NA    NA    NA
## 4      7   108   108   108   108   108   108    NA    NA    NA
## 5      8   135   135   133   132   132   133   135    NA    NA
## 6      9    84    83    82    82    83    84    82    81    NA
## 7     10    68    67    67    68    66    68    68    68    67

```

```

obs_linetypes <- c("solid", "solid", "solid", "solid",
                  "solid", "solid", "solid", "solid", "solid")

```

```
pred_linetypes <- "dashed"
```

```
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
```

```
pos_len_summary$pos <- factor(pos_len_summary$pos)
```

```
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
```

```

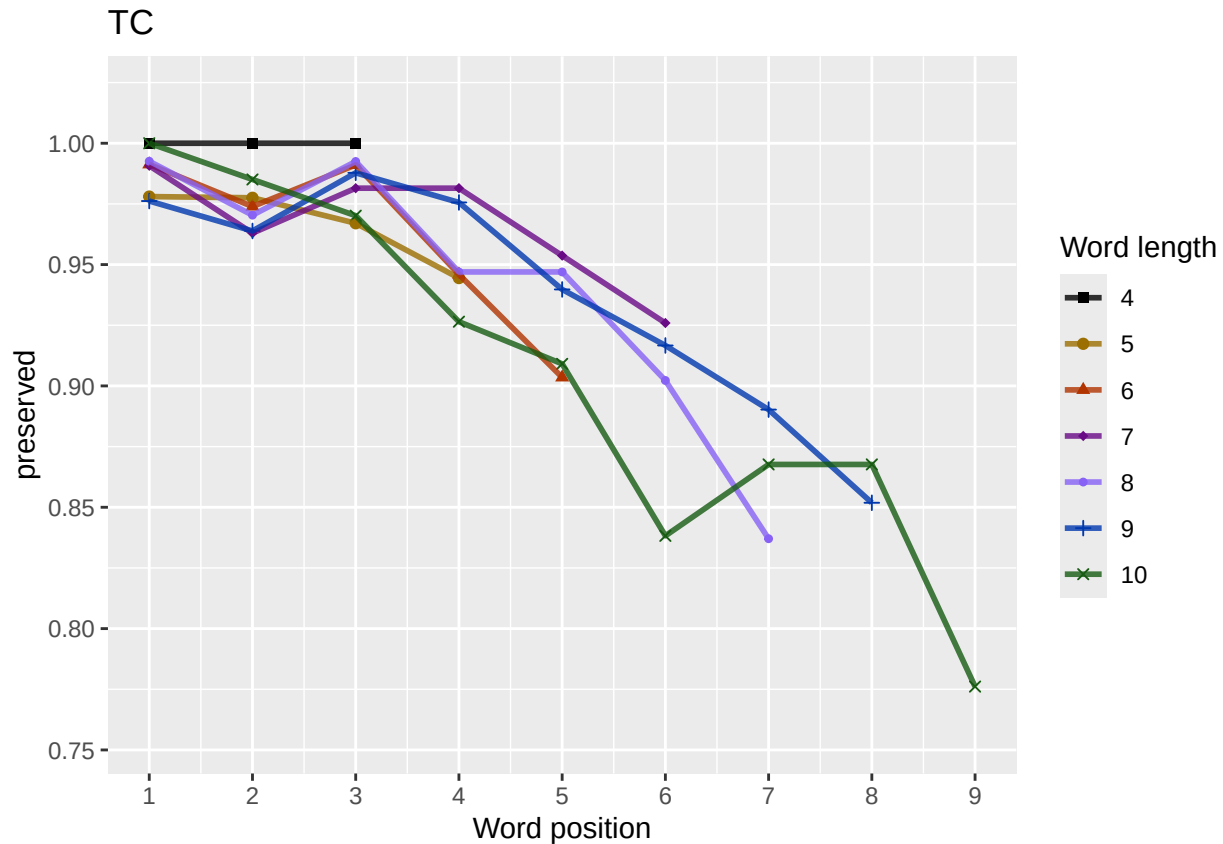
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                           paste0(CurPat),
                                           NULL,
                                           c(min_preserved, max_preserved),
                                           palette_values,
                                           shape_values,
                                           obs_linetypes,
                                           pred_linetypes)

```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## *****
```

```
## model index: 3
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```

##
## Coefficients:
## (Intercept)          pos
##      4.7096      -0.3971
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3951 Residual
## Null Deviance:      1641
## Residual Deviance: 1494 AIC: 1498
## log likelihood:  -747.2054
## Nagelkerke R2:  0.1074116
## % pres/err predicted correctly:  -379.2816
## % of predictable range [ (model-null)/(1-null) ]:  0.04376851
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      5.06673      0.01615     -0.56366
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3950 Residual
## Null Deviance:      1641
## Residual Deviance: 1493 AIC: 1499
## log likelihood:  -746.6254
## Nagelkerke R2:  0.1082436
## % pres/err predicted correctly:  -379.6487
## % of predictable range [ (model-null)/(1-null) ]:  0.04284553
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      4.86750     -0.02639     -0.38639
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3950 Residual
## Null Deviance:      1641
## Residual Deviance: 1494 AIC: 1500
## log likelihood:  -747.0982
## Nagelkerke R2:  0.1075654
## % pres/err predicted correctly:  -379.2315
## % of predictable range [ (model-null)/(1-null) ]:  0.04389452
## *****
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos

```

```

##      5.34249      -0.03917      0.01803      -0.56713
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3949 Residual
## Null Deviance:      1641
## Residual Deviance: 1493  AIC: 1501
## log likelihood:  -746.3979
## Nagelkerke R2:  0.1085699
## % pres/err predicted correctly:  -379.616
## % of predictable range [ (model-null)/(1-null) ]:  0.04292765
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##      5.86229      -0.14430      -0.62515      0.02742
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3949 Residual
## Null Deviance:      1641
## Residual Deviance: 1493  AIC: 1501
## log likelihood:  -746.4594
## Nagelkerke R2:  0.1084817
## % pres/err predicted correctly:  -379.447
## % of predictable range [ (model-null)/(1-null) ]:  0.04335255
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)          pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      4.717032      0.017733      -0.054305      -0.070068      0.007339      -0.070068
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3947 Residual
## Null Deviance:      1641
## Residual Deviance: 1492  AIC: 1504
## log likelihood:  -746.0795
## Nagelkerke R2:  0.1090264
## % pres/err predicted correctly:  -379.5466
## % of predictable range [ (model-null)/(1-null) ]:  0.04310224
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      5.0845      -0.2786
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3951 Residual

```



```
## Null Deviance:      1641
## Residual Deviance: 1603 AIC: 1607
## log likelihood:    -801.2893
## Nagelkerke R2:     0.02875363
## % pres/err predicted correctly: -392.8244
## % of predictable range [ (model-null)/(1-null) ]:  0.009714795
## *****
## model index:      1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)
##          2.881
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3952 Residual
## Null Deviance:      1641
## Residual Deviance: 1641 AIC: 1643
## log likelihood:    -820.6962
## Nagelkerke R2:     -6.534383e-16
## % pres/err predicted correctly: -396.6878
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                             "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ pos	1498.41	0.000000	0.000000	0.368746	0.210741	1.467095	85 NA	-	NA	NA	NA
								0.3970868			
preserved ~	1499.25	0.839976	0.657050	0.422864	0.108243	0.66732	NA	-	NA	0.0161532	NA
I(pos^2) + pos								0.5636646			
preserved ~	1500.19	1.785551	0.409510	0.515100	0.010756	1.486750	4 -	-	NA	NA	NA
stimlen + pos								0.026390	0.3863883		
preserved ~	1500.79	2.384893	0.303470	0.811190	0.031085	0.934248	6 -	-	NA	0.0180325	NA
stimlen + I(pos^2)								0.039165	0.5671344		
+ pos											

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * pos	1500.912	1507.919	190.28537	0.0610523	0.1108481	57862288	-	-	0.0274219	NA	NA
							0.144301	0.126251	0.1489		
preserved ~ stimlen *	1504.153	1507.480	966.05646	0.0902082	0.0109024	447170320	0.0177328	-	-	-	0.0073386
									0.070067	0.0490168	0.0543045
(I(pos^2) + pos)											
preserved ~ stimlen	1606.570	1608.167	770000000	0.0000000	0.0000000	2875360	84536	-	NA	NA	NA
									0.2785772		
preserved ~ 1	1643.392	1644.981	503000000	0.0000000	0.0000000	20880535	NA	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ pos"
```

```
print(BestLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept)          pos
```

```
##      4.7096      -0.3971
```

```
##
```

```
## Degrees of Freedom: 3952 Total (i.e. Null); 3951 Residual
```

```
## Null Deviance: 1641
```

```
## Residual Deviance: 1494 AIC: 1498
```

```
PosDat$LPFitted<-fitted(BestLPModel)
```

```
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],  
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
```

```
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
```

```
## # Groups:   stimlen [7]
```

```
##   stimlen  `1`  `2`  `3`  `4`  `5`  `6`  `7`  `8`  `9`  
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
```

```
## 1      4 0.987 0.980 0.971 NA      NA      NA      NA      NA      NA
```

```
## 2      5 0.987 0.980 0.971 0.958 NA      NA      NA      NA      NA
```

```
## 3      6 0.987 0.980 0.971 0.958 0.938 NA      NA      NA      NA
```

```
## 4      7 0.987 0.980 0.971 0.958 0.938 0.911 NA      NA      NA
```

```
## 5      8 0.987 0.980 0.971 0.958 0.938 0.911 0.873 NA      NA
```

```
## 6      9 0.987 0.980 0.971 0.958 0.938 0.911 0.873 0.822 NA
```

```
## 7     10 0.987 0.980 0.971 0.958 0.938 0.911 0.873 0.822 0.757
```

```

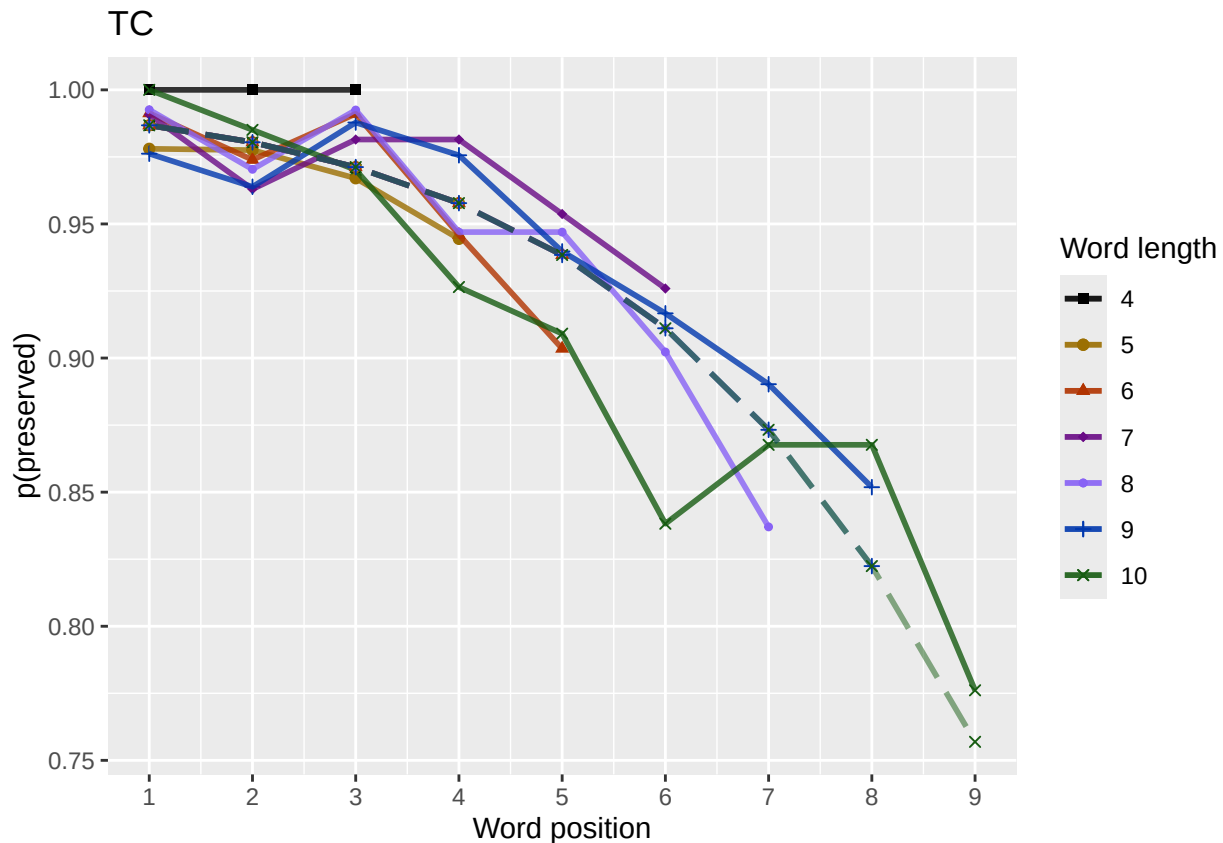
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen))
# geom_point(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen)) + ggtitle(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"))

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot

```



length and position without fragments to see if this changes position² influence

```

# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

```

```

# number responses
resp_num<-0
prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1      26  660

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
              num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 26 / 660 = 3.94 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##    4.7222    -0.3267
##
## Degrees of Freedom: 3884 Total (i.e. Null); 3883 Residual
## Null Deviance: 1219
## Residual Deviance: 1149 AIC: 1153
## log likelihood: -574.4505
## Nagelkerke R2: 0.06596514
## % pres/err predicted correctly: -266.969
## % of predictable range [ (model-null)/(1-null) ]: 0.02065108

```

```

## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##    4.49789      0.03784     -0.34205
##
## Degrees of Freedom: 3884 Total (i.e. Null); 3882 Residual
## Null Deviance:      1219
## Residual Deviance: 1149 AIC: 1155
## log likelihood: -574.283
## Nagelkerke R2: 0.06627966
## % pres/err predicted correctly: -266.9446
## % of predictable range [ (model-null)/(1-null) ]: 0.02074046
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##    4.918812      0.009709     -0.423623
##
## Degrees of Freedom: 3884 Total (i.e. Null); 3882 Residual
## Null Deviance:      1219
## Residual Deviance: 1149 AIC: 1155
## log likelihood: -574.2953
## Nagelkerke R2: 0.06625656
## % pres/err predicted correctly: -267.0304
## % of predictable range [ (model-null)/(1-null) ]: 0.02042673
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos stimlen:pos
##    5.57103      -0.09169     -0.61218      0.03149
##
## Degrees of Freedom: 3884 Total (i.e. Null); 3881 Residual
## Null Deviance:      1219
## Residual Deviance: 1147 AIC: 1155
## log likelihood: -573.6454
## Nagelkerke R2: 0.06747692
## % pres/err predicted correctly: -266.9252
## % of predictable range [ (model-null)/(1-null) ]: 0.02081134
## *****
## model index: 7
##

```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
## 4.696050      0.032078      0.008096      -0.420560
##
## Degrees of Freedom: 3884 Total (i.e. Null); 3881 Residual
## Null Deviance: 1219
## Residual Deviance: 1148 AIC: 1156
## log likelihood: -574.1793
## Nagelkerke R2: 0.06647444
## % pres/err predicted correctly: -266.9992
## % of predictable range [ (model-null)/(1-null) ]: 0.02054097
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos stimlen:I(pos^2)      stiml
## 4.20852      0.04728      -0.11330      0.25436      0.01162      -0
##
## Degrees of Freedom: 3884 Total (i.e. Null); 3879 Residual
## Null Deviance: 1219
## Residual Deviance: 1146 AIC: 1158
## log likelihood: -573.14
## Nagelkerke R2: 0.0684257
## % pres/err predicted correctly: -266.7203
## % of predictable range [ (model-null)/(1-null) ]: 0.02156026
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
## 4.6682      -0.1794
##
## Degrees of Freedom: 3884 Total (i.e. Null); 3883 Residual
## Null Deviance: 1219
## Residual Deviance: 1207 AIC: 1211
## log likelihood: -603.4108
## Nagelkerke R2: 0.01116405
## % pres/err predicted correctly: -271.8221
## % of predictable range [ (model-null)/(1-null) ]: 0.002914523
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##

```

```

## Coefficients:
## (Intercept)
##      3.272
##
## Degrees of Freedom: 3884 Total (i.e. Null);  3884 Residual
## Null Deviance:      1219
## Residual Deviance: 1219  AIC: 1221
## log likelihood:  -609.258
## Nagelkerke R2:  -8.247623e-16
## % pres/err predicted correctly:  -272.6196
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                                AIC=NoFrag_LPRes$AIC,
                                row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ pos	1152.901	0.000000	0.000000	0.000000	0.0414472	0.0659631	1722190	NA	-	NA	NA
									0.3266713		
preserved ~	1154.566	1.665070	0.434940	0.518027	0.0662797	0.4978880	0.378447	-	NA	NA	NA
stimlen + pos									0.3420500		
preserved ~	1154.591	1.689670	0.429627	0.317806	0.0706625	0.0918812	NA	-	NA	0.0097085	NA
I(pos^2) + pos									0.4236233		
preserved ~	1155.292	2.389860	0.302725	0.212547	0.0306747	0.69571030	-	-	0.0314871	NA	NA
stimlen * pos									0.0916938	0.6121820	
preserved ~	1156.353	3.457640	0.177493	0.070735	0.0663066	0.474469	0.60500	0.320779	-	NA	0.0080961
stimlen + I(pos^2)									0.4205604		
+ pos											
preserved ~	1158.280	5.379010	0.067910	0.030281	0.0068423	0.72085190	0.0472810	0.2543645	-	-	0.0116155
stimlen * (I(pos^2)									0.0568870	0.1132987	
+ pos)											
preserved ~	1210.822	7.920508	0.000000	0.000000	0.000000	0.111640	0.668160	-	NA	NA	NA
stimlen									0.1794096		
preserved ~ 1	1220.516	7.614897	0.000000	0.000000	0.000000	0.000000	0.0271816	NA	NA	NA	NA

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

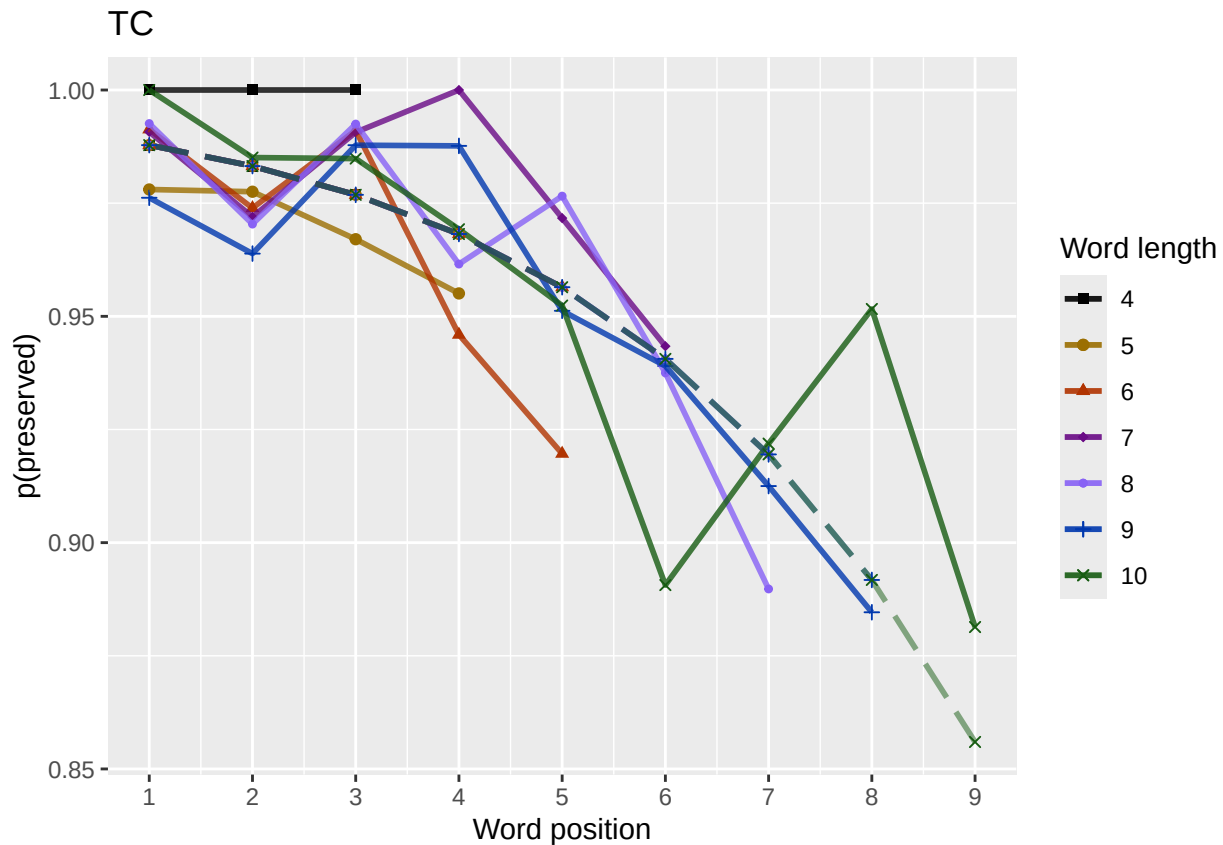
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.988 0.983 0.977 NA      NA      NA      NA      NA      NA
## 2      5 0.988 0.983 0.977 0.968 NA      NA      NA      NA      NA
## 3      6 0.988 0.983 0.977 0.968 0.956 NA      NA      NA      NA
## 4      7 0.988 0.983 0.977 0.968 0.956 0.941 NA      NA      NA
## 5      8 0.988 0.983 0.977 0.968 0.956 0.941 0.919 NA      NA
## 6      9 0.988 0.983 0.977 0.968 0.956 0.941 0.919 0.892 NA
## 7     10 0.988 0.983 0.977 0.968 0.956 0.941 0.919 0.892 0.856
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen))
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("no_fragments_percent_preserved_by_length_pos_wfit.png"))
```

```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.75 - 1.02"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] 0
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] -0.02873359
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
                              2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "No U-shape in this participant"
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentage_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentage_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "no U-shape in this participant"

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small sample sizes)
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##

```

```

## Coefficients:
## (Intercept)          pos      log_freq
##      4.6962      -0.3796      0.1888
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3950 Residual
## Null Deviance:      1641
## Residual Deviance: 1471 AIC: 1477
## log likelihood:  -735.5373
## Nagelkerke R2:  0.124101
## % pres/err predicted correctly:  -375.6554
## % of predictable range [ (model-null)/(1-null) ]:  0.05288689
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos      log_freq
##      4.51380      0.02998     -0.39095      0.19291
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3949 Residual
## Null Deviance:      1641
## Residual Deviance: 1471 AIC: 1479
## log likelihood:  -735.4068
## Nagelkerke R2:  0.1242871
## % pres/err predicted correctly:  -375.6218
## % of predictable range [ (model-null)/(1-null) ]:  0.05297136
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq          pos  stimlen:log_freq
##      4.61166      0.01549      0.48596     -0.39020     -0.03566
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3948 Residual
## Null Deviance:      1641
## Residual Deviance: 1469 AIC: 1479
## log likelihood:  -734.4562
## Nagelkerke R2:  0.1256424
## % pres/err predicted correctly:  -375.8631
## % of predictable range [ (model-null)/(1-null) ]:  0.0523644
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq  pos:log_freq
##      4.70971     -0.38240      0.21039     -0.00404

```

```

##
## Degrees of Freedom: 3952 Total (i.e. Null); 3949 Residual
## Null Deviance: 1641
## Residual Deviance: 1471 AIC: 1479
## log likelihood: -735.5126
## Nagelkerke R2: 0.1241363
## % pres/err predicted correctly: -375.7394
## % of predictable range [ (model-null)/(1-null) ]: 0.05267554
## *****
## model index: 13
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq
## 4.94646 0.01792 0.01639 -0.55480 0.19170
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3948 Residual
## Null Deviance: 1641
## Residual Deviance: 1470 AIC: 1480
## log likelihood: -734.8358
## Nagelkerke R2: 0.1251012
## % pres/err predicted correctly: -375.9431
## % of predictable range [ (model-null)/(1-null) ]: 0.05216344
## *****
## model index: 11
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq I(pos^2) pos stimlen:log_freq
## 5.019327 0.004254 0.479030 0.015477 -0.544921 -0.01056
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3947 Residual
## Null Deviance: 1641
## Residual Deviance: 1468 AIC: 1480
## log likelihood: -733.9461
## Nagelkerke R2: 0.1263693
## % pres/err predicted correctly: -376.1608
## % of predictable range [ (model-null)/(1-null) ]: 0.05161584
## *****
## model index: 9
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos log_freq I(pos^2):log_freq
## 4.95821 0.01059 -0.49381 -0.02833 -0.01056
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3947 Residual
## Null Deviance: 1641

```

```

## Residual Deviance: 1468 AIC: 1480
## log likelihood: -734.0137
## Nagelkerke R2: 0.126273
## % pres/err predicted correctly: -375.8939
## % of predictable range [ (model-null)/(1-null) ]: 0.05228695
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq log_fr
## 4.622335 0.007473 0.499250 -0.380139 -0.045757 0.
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3947 Residual
## Null Deviance: 1641
## Residual Deviance: 1469 AIC: 1481
## log likelihood: -734.2899
## Nagelkerke R2: 0.1258793
## % pres/err predicted correctly: -375.6818
## % of predictable range [ (model-null)/(1-null) ]: 0.05282028
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq pos:log_freq
## 4.517332 0.032408 -0.395582 0.222360 -0.005443
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3948 Residual
## Null Deviance: 1641
## Residual Deviance: 1471 AIC: 1481
## log likelihood: -735.3632
## Nagelkerke R2: 0.1243493
## % pres/err predicted correctly: -375.7309
## % of predictable range [ (model-null)/(1-null) ]: 0.05269704
## *****
## model index: 12
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq I(pos
## 4.811764 0.020740 0.009529 -0.491303 -0.021162
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3946 Residual
## Null Deviance: 1641
## Residual Deviance: 1468 AIC: 1482
## log likelihood: -733.9551
## Nagelkerke R2: 0.1263564

```

```

## % pres/err predicted correctly: -375.8716
## % of predictable range [ (model-null)/(1-null) ]: 0.0523431
## *****
## model index: 10
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen          log_freq          I(pos^2)          pos          stim
## 4.9213915      -0.0008014      0.2544971      0.0105747      -0.4880411
## log_freq:pos
## 0.1054270
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3945 Residual
## Null Deviance: 1641
## Residual Deviance: 1466 AIC: 1482
## log likelihood: -733.1543
## Nagelkerke R2: 0.1274974
## % pres/err predicted correctly: -375.8714
## % of predictable range [ (model-null)/(1-null) ]: 0.05234376
## *****
## model index: 16
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
## 4.7096      -0.3971
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3951 Residual
## Null Deviance: 1641
## Residual Deviance: 1494 AIC: 1498
## log likelihood: -747.2054
## Nagelkerke R2: 0.1074116
## % pres/err predicted correctly: -379.2816
## % of predictable range [ (model-null)/(1-null) ]: 0.04376851
## *****
## model index: 19
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          I(pos^2)          pos
## 5.06673      0.01615      -0.56366
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3950 Residual
## Null Deviance: 1641
## Residual Deviance: 1493 AIC: 1499
## log likelihood: -746.6254
## Nagelkerke R2: 0.1082436
## % pres/err predicted correctly: -379.6487

```



```

## % of predictable range [ (model-null)/(1-null) ]: 0.04284553
## *****
## model index: 17
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##    4.86750    -0.02639    -0.38639
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3950 Residual
## Null Deviance:      1641
## Residual Deviance: 1494 AIC: 1500
## log likelihood: -747.0982
## Nagelkerke R2: 0.1075654
## % pres/err predicted correctly: -379.2315
## % of predictable range [ (model-null)/(1-null) ]: 0.04389452
## *****
## model index: 20
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##    5.34249    -0.03917    0.01803    -0.56713
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3949 Residual
## Null Deviance:      1641
## Residual Deviance: 1493 AIC: 1501
## log likelihood: -746.3979
## Nagelkerke R2: 0.1085699
## % pres/err predicted correctly: -379.616
## % of predictable range [ (model-null)/(1-null) ]: 0.04292765
## *****
## model index: 18
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos stimlen:pos
##    5.86229    -0.14430    -0.62515    0.02742
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3949 Residual
## Null Deviance:      1641
## Residual Deviance: 1493 AIC: 1501
## log likelihood: -746.4594
## Nagelkerke R2: 0.1084817
## % pres/err predicted correctly: -379.447
## % of predictable range [ (model-null)/(1-null) ]: 0.04335255
## *****
## model index: 21

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen          I(pos^2)          pos  stimlen:I(pos^2)          stimlen:I(pos^2)
## 4.717032          0.017733          -0.054305          -0.070068          0.007339          -0.070068
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3947 Residual
## Null Deviance: 1641
## Residual Deviance: 1492 AIC: 1504
## log likelihood: -746.0795
## Nagelkerke R2: 0.1090264
## % pres/err predicted correctly: -379.5466
## % of predictable range [ (model-null)/(1-null) ]: 0.04310224
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen          log_freq  stimlen:log_freq
## 4.84907          -0.24202          0.48758          -0.03692
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3949 Residual
## Null Deviance: 1641
## Residual Deviance: 1579 AIC: 1587
## log likelihood: -789.2876
## Nagelkerke R2: 0.04639487
## % pres/err predicted correctly: -390.6347
## % of predictable range [ (model-null)/(1-null) ]: 0.01522086
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen          log_freq
## 4.7454          -0.2272          0.1831
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3950 Residual
## Null Deviance: 1641
## Residual Deviance: 1581 AIC: 1587
## log likelihood: -790.3349
## Nagelkerke R2: 0.04485973
## % pres/err predicted correctly: -390.4156
## % of predictable range [ (model-null)/(1-null) ]: 0.01577164
## *****
## model index: 15
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)

```

```
##
## Coefficients:
## (Intercept)      stimlen
##      5.0845      -0.2786
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3951 Residual
## Null Deviance:      1641
## Residual Deviance: 1603  AIC: 1607
## log likelihood:  -801.2893
## Nagelkerke R2:  0.02875363
## % pres/err predicted correctly:  -392.8244
## % of predictable range [ (model-null)/(1-null) ]:  0.009714795
## *****
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.881
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3952 Residual
## Null Deviance:      1641
## Residual Deviance: 1641  AIC: 1643
## log likelihood:  -820.6962
## Nagelkerke R2:  -6.534383e-16
## % pres/err predicted correctly:  -396.6878
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                           AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                       by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_freq_len_pos_model_summary.csv"),row.names = FALSE)

kable(FLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	log_stimlen	log_freq	log_pos	log_freq_pos	log_freq_pos^2	log_freq_pos^3	log_freq_pos^4	log_freq_pos^5	len:I(pos^2)
preserved ~	1477.075	0.0000000	0.0000000	0.0000000	0.0000000	2.881	0.1887817	-	NA	NA	NA	NA	NA	NA	NA
pos +								0.3796488							
log_freq															

Model	AIC Delta	AIC	AICw	NagR ²	Intercept	log_stimlen	log_freq	log_pos	log_freq(I(pos^2))	log_freq(I(pos^2) + pos)	log_freq(I(pos^2) + pos) *	log_freq(I(pos^2) + pos) *	log_freq(I(pos^2) + pos) *
preserved ~ stimlen + pos + log_freq	1478.874	1490.141	190.371	0.288	1.351	138002908	0.291	NA	NA	NA	NA	NA	NA
preserved ~ stimlen *	1478.182	1782398	0.529	0.572	6.424	66315088	79596	-	NA	NA	NA	NA	NA
log_freq + pos	1479.125	06087	70.770	0.936	1.369	NA	0.210	3948	-	-	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos + log_freq	1479.172	70227	20233	945	4.916	4591792	38170	0.5	NA	NA	0.016	3808	NA
preserved ~ stimlen *	1479.282	76874	44258	0.146	3.613	3270405	47902	97	-	NA	NA	0.015	4767
log_freq + I(pos^2) + pos	1480.127	20828	46066	606	2.738	NA	-	NA	-	0.105	8784	0.010	5949
preserved ~ stimlen + log_freq	1480.380	50157	33129	0.175	8.722	3350707	49924	99	-	NA	0.012	3606	NA
preserved ~ stimlen *	1480.726	18061	07146	562	4.517	3323008	23603	-	-	NA	NA	NA	NA
log_freq + pos *	1481.483	50708	90025	0.226	3.814	7692073	95	NA	-	0.105	2861	0.009	5287
log_freq + I(pos^2) + pos *	1482.503	40297	30206	0.427	4.921	392	0.254	4971	-	NA	0.105	4200	5747
preserved ~ stimlen + log_freq	1498.211	33637	00233	0.067	4.709	581	NA	NA	-	NA	NA	NA	NA
preserved ~ I(pos^2) + pos	1499.221	76330	00163	0.048	2.086	732	NA	NA	-	NA	NA	0.016	1532

Model	AIC Delta	AIC	AICw	NagR ²	(Intercept)	log_stimlen	log_pos	log_freq	I(pos^2)	pos^2	log_freq	I(pos^2)	pos^2	log_freq	I(pos^2)	pos^2
preserved ~ stimlen + pos	1500.236209	2000.000000	0.000000	0.287504	NA	NA	-	NA	NA	NA	NA	NA	NA	NA	NA	NA
				0.0263900		0.3863883										
preserved ~ stimlen + I(pos^2) + pos	1500.236209	2000.000000	0.000000	0.287504	NA	NA	-	NA	NA	0.0180325	NA	NA	NA	NA	NA	NA
				0.0391656		0.5671344										
preserved ~ stimlen * pos	1500.236209	2000.000000	0.000000	0.287504	NA	NA	-	NA	NA	NA	NA	NA	NA	0.0274210	NA	NA
				0.1443012		0.6251489										
preserved ~ stimlen * (I(pos^2) + pos)	1504.279804	2000.000000	0.000000	0.261703	NA	NA	-	NA	NA	-	NA	NA	-	0.0073386	0.0490168	NA
				0.0700679		0.0543045										
preserved ~ stimlen * log_freq	1586.579500	2000.000000	0.000000	0.163849	0.4875753	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
				0.2420223	0.0369224											
preserved ~ stimlen + log_freq	1586.579500	2000.000000	0.000000	0.163849	0.1830544	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
				0.2272350												
preserved ~ stimlen	1606.529504	2000.000000	0.000000	0.153861	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
				0.2785772												
preserved ~ 1	1643.392307	2000.000000	0.000000	0.080533	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ pos + log_freq"
```

```
print(BestFLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos      log_freq
##      4.6962      -0.3796      0.1888
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3950 Residual
## Null Deviance:      1641
## Residual Deviance: 1471 AIC: 1477
```

```
# do a median split on frequency to plot hf/ls effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq] <- "hf"
PosDat$freq_bin[PosDat$log_freq < median_freq] <- "lf"
```

```
PosDat$FLPFitted <- fitted(BestFLPModel)
```

```
HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]
```

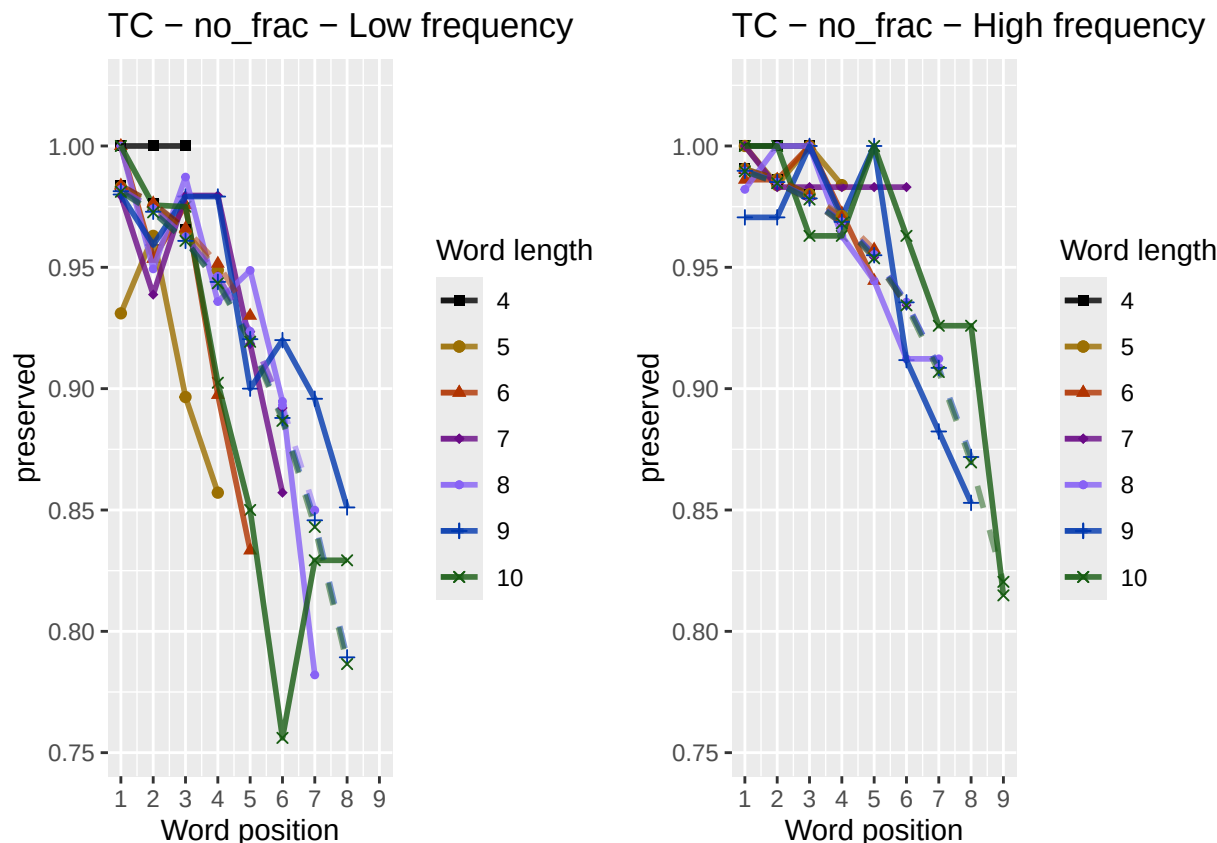
```
HF_Plot <- plot_len_pos_obs_predicted(HFDat, paste0(CurPat, " - ", RunLabel, " - High frequency"), "FLPFitted")
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
LF_Plot <- plot_len_pos_obs_predicted(LFdat, paste0(CurPat, " - ", RunLabel, " - Low frequency"), "FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
library(ggpubr)
Both_Plots <- ggarrange(LF_Plot, HF_Plot) # labels=c("LF", "HF", ncol=2)

## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_line()`).
## Removed 1 row containing missing values or values outside the scale range (`geom_line()`).
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_",
              CurTask, "_frequency_effect_length_pos_wfit.png"),
       device="png", unit="cm", width=30, height=11)
print(Both_Plots)
```



```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
```

```

"preserved ~ (I(pos^2)+pos)",
"preserved ~ pos",
"preserved ~ stimlen",
"preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.501      -2.588
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3951 Residual
## Null Deviance:      1641
## Residual Deviance: 1198 AIC: 1202
## log likelihood: -598.902
## Nagelkerke R2: 0.3123765
## % pres/err predicted correctly: -281.957
## % of predictable range [ (model-null)/(1-null) ]: 0.2884947
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      4.7096      -0.3971
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3951 Residual
## Null Deviance:      1641
## Residual Deviance: 1494 AIC: 1498
## log likelihood: -747.2054
## Nagelkerke R2: 0.1074116
## % pres/err predicted correctly: -379.2816
## % of predictable range [ (model-null)/(1-null) ]: 0.04376851
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      5.06673      0.01615      -0.56366
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3950 Residual
## Null Deviance:      1641
## Residual Deviance: 1493 AIC: 1499

```

```

## log likelihood: -746.6254
## Nagelkerke R2: 0.1082436
## % pres/err predicted correctly: -379.6487
## % of predictable range [ (model-null)/(1-null) ]: 0.04284553
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 5.0845 -0.2786
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3951 Residual
## Null Deviance: 1641
## Residual Deviance: 1603 AIC: 1607
## log likelihood: -801.2893
## Nagelkerke R2: 0.02875363
## % pres/err predicted correctly: -392.8244
## % of predictable range [ (model-null)/(1-null) ]: 0.009714795
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 3.2602 -0.1295
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3951 Residual
## Null Deviance: 1641
## Residual Deviance: 1626 AIC: 1630
## log likelihood: -813.1769
## Nagelkerke R2: 0.0111743
## % pres/err predicted correctly: -395.1415
## % of predictable range [ (model-null)/(1-null) ]: 0.003888345
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.881
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3952 Residual
## Null Deviance: 1641
## Residual Deviance: 1641 AIC: 1643
## log likelihood: -820.6962
## Nagelkerke R2: -6.534383e-16
## % pres/err predicted correctly: -396.6878

```



```
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                             CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1201.8040	0.0000	1	1	0.3123763	5.501229	NA	-	NA	NA	NA
preserved ~ pos	1498.412	296.6070	0	0	0.1074114	6.709585	NA	NA	NA	-	NA
										0.3970868	
preserved ~ (I(pos^2) + pos)	1499.252	297.4470	0	0	0.1082435	6.066732	NA	NA	0.0161532	-	NA
										0.5636646	
preserved ~ stimlen	1606.5794	404.7748	0	0	0.0287535	6.084536	NA	NA	NA	NA	-
											0.2785772
preserved ~ CumPres	1630.354	428.5499	0	0	0.0111743	2.260193	-	NA	NA	NA	NA
							0.1295356				
preserved ~ 1	1643.3924	441.5885	0	0	0.0000000	0.880535	NA	NA	NA	NA	NA

```
if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                      family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
```

```

        rep(BestMEMModelFormulaRnd, RandomSamples))
AICValues <- c(BestMEMModel$aic, RndModelAIC)
BestMEMModelRndDF <- data.frame(Name=ModelNames, AIC=AICValues)
BestMEMModelRndDF <- BestMEMModelRndDF %>% arrange(AIC)
BestMEMModelRndDF <- rbind(BestMEMModelRndDF,
                           data.frame(Name=c("Random average"),
                                       AIC=c(mean(RndModelAIC))))
BestMEMModelRndDF <- rbind(BestMEMModelRndDF,
                           data.frame(Name=c("Random SD"),
                                       AIC=c(sd(RndModelAIC))))

write.csv(BestMEMModelRndDF,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_best_main_effects_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary, paste0(TablesDir, CurPat, "_",
                                         RunLabel, "_", CurTask,
                                         "_syllable_component_summary.csv"), row.names = FALSE)
kable(syll_component_summary)

```

syll_component	MeanPres	N
1	0.9503817	524
O	0.9423394	1821
P	0.9714286	35
S	0.9191489	235
V	0.9559043	1338

```

# main effects models for data without satellite positions

keep_components = c("0", "V", "1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen, stim, pos,
                             preserved, syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data, MEModelEquations)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.513      -2.796

```

```

##
## Degrees of Freedom: 3682 Total (i.e. Null); 3681 Residual
## Null Deviance: 1497
## Residual Deviance: 1094 AIC: 1098
## log likelihood: -547.2318
## Nagelkerke R2: 0.3097169
## % pres/err predicted correctly: -256.7586
## % of predictable range [ (model-null)/(1-null) ]: 0.2847915
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 4.7129 -0.3895
##
## Degrees of Freedom: 3682 Total (i.e. Null); 3681 Residual
## Null Deviance: 1497
## Residual Deviance: 1368 AIC: 1372
## log likelihood: -684.1062
## Nagelkerke R2: 0.1025358
## % pres/err predicted correctly: -344.6781
## % of predictable range [ (model-null)/(1-null) ]: 0.04083914
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 5.07486 0.01632 -0.55801
##
## Degrees of Freedom: 3682 Total (i.e. Null); 3680 Residual
## Null Deviance: 1497
## Residual Deviance: 1367 AIC: 1373
## log likelihood: -683.5686
## Nagelkerke R2: 0.10338
## % pres/err predicted correctly: -345.0269
## % of predictable range [ (model-null)/(1-null) ]: 0.03987143
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 5.1716 -0.2851
##
## Degrees of Freedom: 3682 Total (i.e. Null); 3681 Residual
## Null Deviance: 1497

```

```
## Residual Deviance: 1460 AIC: 1464
## log likelihood: -729.8922
## Nagelkerke R2: 0.02972246
## % pres/err predicted correctly: -355.8072
## % of predictable range [ (model-null)/(1-null) ]: 0.009959146
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 3.2139 -0.1118
##
## Degrees of Freedom: 3682 Total (i.e. Null); 3681 Residual
## Null Deviance: 1497
## Residual Deviance: 1487 AIC: 1491
## log likelihood: -743.5908
## Nagelkerke R2: 0.007583505
## % pres/err predicted correctly: -358.4698
## % of predictable range [ (model-null)/(1-null) ]: 0.002571213
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.911
##
## Degrees of Freedom: 3682 Total (i.e. Null); 3682 Residual
## Null Deviance: 1497
## Residual Deviance: 1497 AIC: 1499
## log likelihood: -748.2598
## Nagelkerke R2: 9.974727e-16
## % pres/err predicted correctly: -359.3964
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
write.csv(SimpSyllMEAICSummary,
  paste0(TablesDir, CurPat, "_", RunLabel, "_",
    CurTask,
    "_OV1_main_effects_model_summary.csv"),
  row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1098.464	0.0000	1	1	0.3097169	512933	NA	- 2.796355	NA	NA	NA
preserved ~ pos	1372.212	273.7488	0	0	0.1025358	8.712878	NA	NA	NA	- 0.3895213	NA

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ (I(pos^2) + pos)	1373.137	274.6736	0	0	0.1033806	0.074860	NA	NA	0.0163162	-	NA
preserved ~ stimlen	1463.784	65.3208	0	0	0.0297225	0.171588	NA	NA	NA	NA	-
preserved ~ CumPres	1491.183	92.7179	0	0	0.0075835	0.213864	-	NA	NA	NA	0.2851206
preserved ~ 1	1498.520	100.0558	0	0	0.0000000	0.911492	0.1118052	NA	NA	NA	NA

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0","V")
OVDData <- PosDat[PosDat$syll_component %in% keep_components,]
OVDData <- OVDData %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OVDData$CumPres <- CalcCumPres(OVDData)
OVDData$CumErr <- CalcCumErrFromPreserved(OVDData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVDData,MEModelEquations)
```

```
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.414      -2.900
##
## Degrees of Freedom: 3158 Total (i.e. Null); 3157 Residual
## Null Deviance:      1290
## Residual Deviance: 1000 AIC: 1004
## log likelihood: -499.9865
## Nagelkerke R2: 0.261378
## % pres/err predicted correctly: -236.8816
## % of predictable range [ (model-null)/(1-null) ]: 0.2350382
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      4.670      -0.381
##
## Degrees of Freedom: 3158 Total (i.e. Null); 3157 Residual
## Null Deviance:      1290
## Residual Deviance: 1178 AIC: 1182
```

```

## log likelihood: -589.0913
## Nagelkerke R2: 0.1033993
## % pres/err predicted correctly: -297.1128
## % of predictable range [ (model-null)/(1-null) ]: 0.04135098
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 4.9277 0.0119 -0.5034
##
## Degrees of Freedom: 3158 Total (i.e. Null); 3156 Residual
## Null Deviance: 1290
## Residual Deviance: 1178 AIC: 1184
## log likelihood: -588.818
## Nagelkerke R2: 0.1038976
## % pres/err predicted correctly: -297.3647
## % of predictable range [ (model-null)/(1-null) ]: 0.04054097
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 5.0316 -0.2695
##
## Degrees of Freedom: 3158 Total (i.e. Null); 3157 Residual
## Null Deviance: 1290
## Residual Deviance: 1261 AIC: 1265
## log likelihood: -630.3499
## Nagelkerke R2: 0.02717593
## % pres/err predicted correctly: -307.1627
## % of predictable range [ (model-null)/(1-null) ]: 0.009033497
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 3.326 -0.180
##
## Degrees of Freedom: 3158 Total (i.e. Null); 3157 Residual
## Null Deviance: 1290
## Residual Deviance: 1273 AIC: 1277
## log likelihood: -636.6506
## Nagelkerke R2: 0.01535935
## % pres/err predicted correctly: -308.2623

```

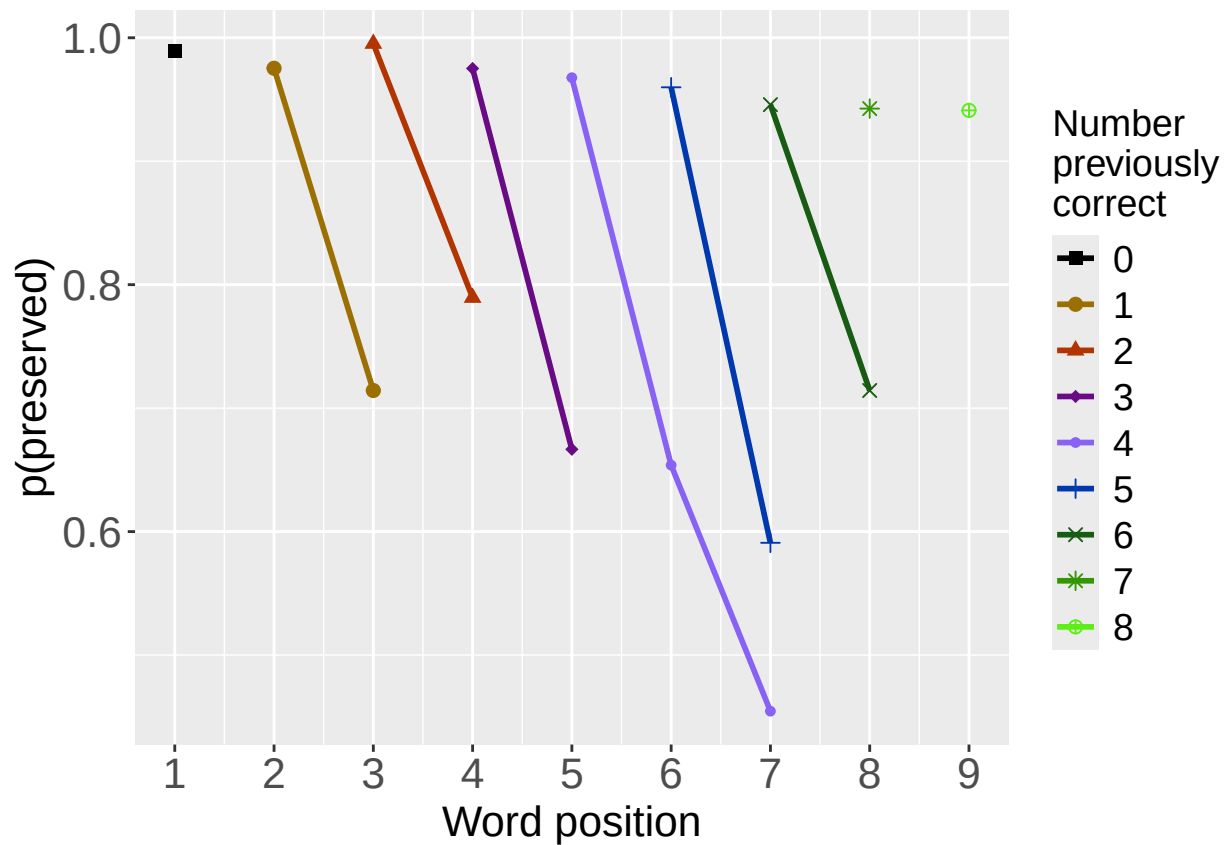
```
## % of predictable range [ (model-null)/(1-null) ]: 0.005497503
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.905
##
## Degrees of Freedom: 3158 Total (i.e. Null); 3158 Residual
## Null Deviance: 1290
## Residual Deviance: 1290 AIC: 1292
## log likelihood: -644.803
## Nagelkerke R2: -6.624724e-16
## % pres/err predicted correctly: -309.9718
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1003.973	0.0000	1	1	0.2613788	0.414127	NA	-	NA	NA	NA
preserved ~ pos	1182.183	178.2097	0	0	0.1033993	1.669509	NA	NA	NA	-	NA
										0.3810461	
preserved ~ (I(pos^2) + pos)	1183.636	179.6631	0	0	0.1038976	1.927666	NA	NA	0.0119017	-	NA
										0.5034016	
preserved ~ stimlen	1264.700	260.7268	0	0	0.0271755	0.031600	NA	NA	NA	NA	-
											0.2694516
preserved ~ CumPres	1277.302	273.3283	0	0	0.0153593	1.326236	-	NA	NA	NA	NA
							0.1799801				
preserved ~ 1	1291.600	287.6330	0	0	0.0000000	0.904833	NA	NA	NA	NA	NA

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

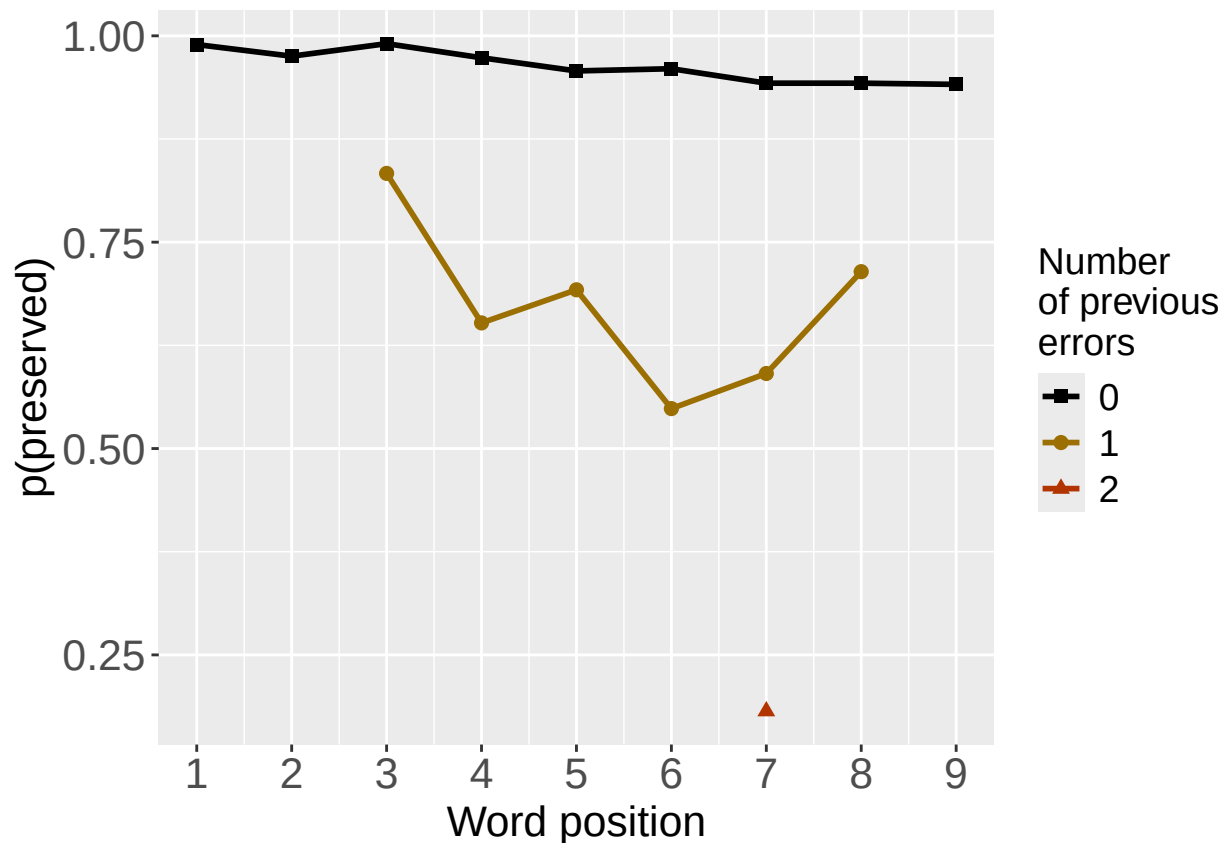
```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

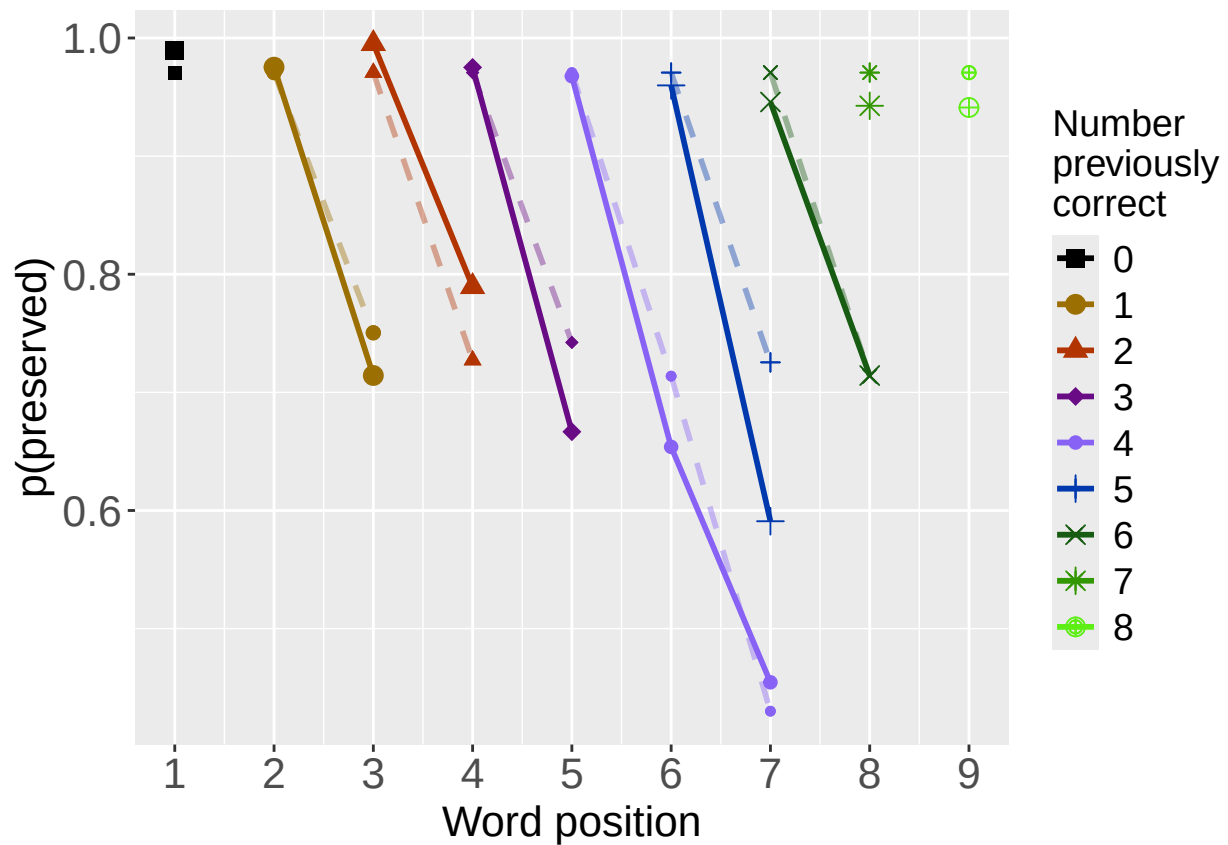
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

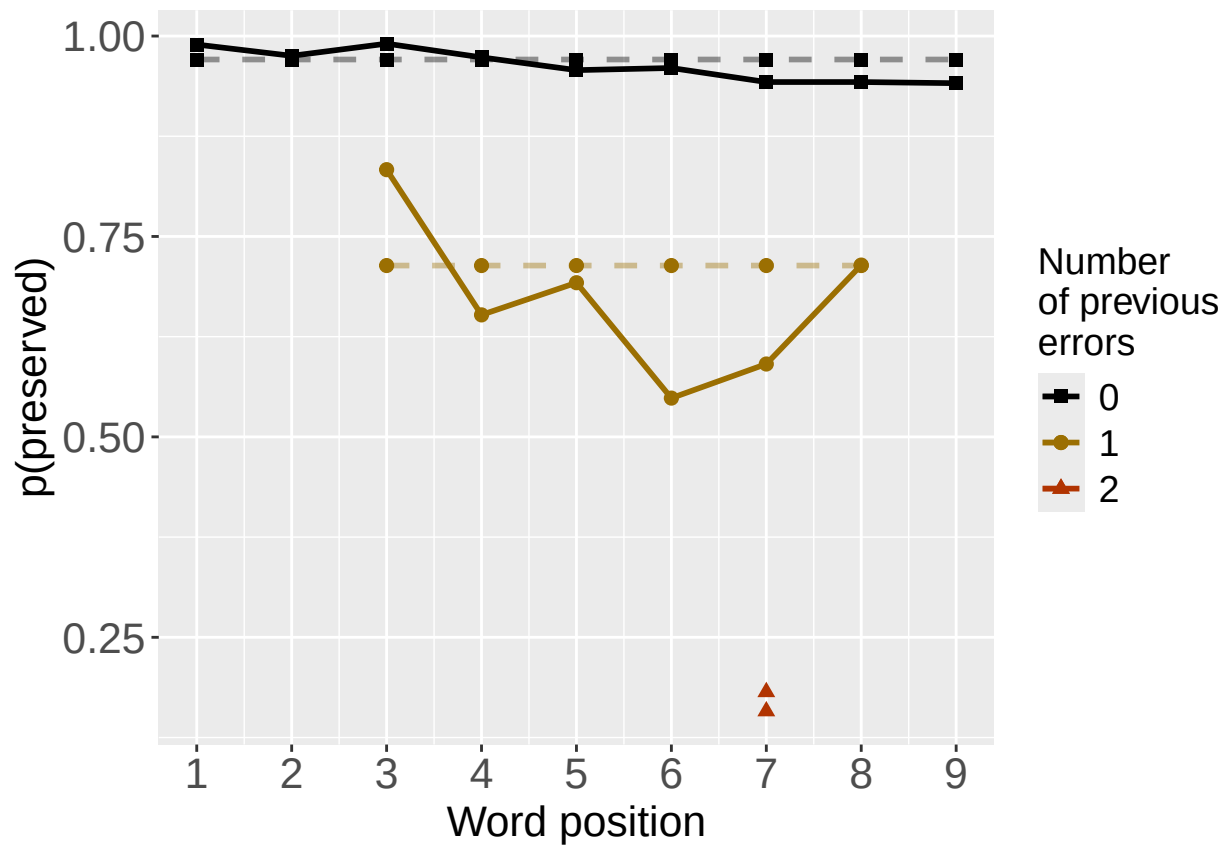
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index: 2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)         pos
##    4.94883    -2.28703     0.02611    -0.48132
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3949 Residual
## Null Deviance:      1641
## Residual Deviance: 1165  AIC: 1173
## log likelihood:  -582.2842
## Nagelkerke R2:  0.3343996
## % pres/err predicted correctly:  -278.7011
## % of predictable range [ (model-null)/(1-null) ]:  0.2966818
## *****
## model index: 1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.501      -2.588
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3951 Residual
## Null Deviance:      1641
## Residual Deviance: 1198  AIC: 1202
## log likelihood:  -598.902
## Nagelkerke R2:  0.3123765
## % pres/err predicted correctly:  -281.957
## % of predictable range [ (model-null)/(1-null) ]:  0.2884947
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      5.06673      0.01615     -0.56366
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3950 Residual
## Null Deviance:      1641
## Residual Deviance: 1493  AIC: 1499
## log likelihood:  -746.6254
## Nagelkerke R2:  0.1082436
## % pres/err predicted correctly:  -379.6487
## % of predictable range [ (model-null)/(1-null) ]:  0.04284553
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	1172.568	0.00000	1e+00	0.9999996	0.3343996	4.948834	-2.287032	0.0261061	-0.4813157
preserved ~ CumErr	1201.804	29.23542	4e-07	0.0000004	0.3123765	3.501229	-2.587645	NA	NA
preserved ~ I(pos^2) + pos	1499.251	326.68238	0e+00	0.0000000	0.1082436	5.066732	NA	0.0161532	-0.5636646

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr stimlen
## 4.5700 -2.5210 -0.1385
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3950 Residual
## Null Deviance: 1641
## Residual Deviance: 1191 AIC: 1197
## log likelihood: -595.4991
## Nagelkerke R2: 0.3169013
## % pres/err predicted correctly: -281.4248
## % of predictable range [ (model-null)/(1-null) ]: 0.2898329
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 3.501 -2.588
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3951 Residual
## Null Deviance: 1641
## Residual Deviance: 1198 AIC: 1202
## log likelihood: -598.902
## Nagelkerke R2: 0.3123765
## % pres/err predicted correctly: -281.957
## % of predictable range [ (model-null)/(1-null) ]: 0.2884947
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      5.0845      -0.2786
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3951 Residual
## Null Deviance:      1641
## Residual Deviance: 1603  AIC: 1607
## log likelihood:  -801.2893
## Nagelkerke R2:   0.02875363
## % pres/err predicted correctly:  -392.8244
## % of predictable range [ (model-null)/(1-null) ]:  0.009714795
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr + stimlen	1196.998	0.000000	1.0000000	0.9170421	0.3169013	4.569992	-	-
preserved ~ CumErr	1201.804	4.805639	0.0904625	0.0829579	0.3123765	3.501229	-	NA
preserved ~ stimlen	1606.579	409.580390	0.0000000	0.0000000	0.0287536	5.084536	NA	-
								0.2785772

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      4.0249      -2.5620      -0.1714
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3950 Residual
## Null Deviance:      1641
## Residual Deviance: 1180  AIC: 1186
## log likelihood:  -589.7951
## Nagelkerke R2:   0.3244685
## % pres/err predicted correctly:  -280.3676
## % of predictable range [ (model-null)/(1-null) ]:  0.2924913
## *****
## model index:  1
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.501      -2.588
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3951 Residual
## Null Deviance:      1641
## Residual Deviance: 1198 AIC: 1202
## log likelihood: -598.902
## Nagelkerke R2: 0.3123765
## % pres/err predicted correctly: -281.957
## % of predictable range [ (model-null)/(1-null) ]: 0.2884947
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      3.2602      -0.1295
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3951 Residual
## Null Deviance:      1641
## Residual Deviance: 1626 AIC: 1630
## log likelihood: -813.1769
## Nagelkerke R2: 0.0111743
## % pres/err predicted correctly: -395.1415
## % of predictable range [ (model-null)/(1-null) ]: 0.003888345
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	1185.590	0.00000	1.0000000	0.9996986	0.3244685	4.024943	- 2.561983	- 0.1713946
preserved ~ CumErr	1201.804	16.21365	0.0003015	0.0003014	0.3123765	3.501229	- 2.587645	NA
preserved ~ CumPres	1630.354	444.76351	0.0000000	0.0000000	0.0111743	3.260193	NA	- 0.1295356

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```



```

## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      4.4419      -2.3013      -0.2256
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3950 Residual
## Null Deviance:      1641
## Residual Deviance: 1167  AIC: 1173
## log likelihood:  -583.3739
## Nagelkerke R2:  0.3329613
## % pres/err predicted correctly:  -278.4147
## % of predictable range [ (model-null)/(1-null) ]:  0.297402
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.501      -2.588
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3951 Residual
## Null Deviance:      1641
## Residual Deviance: 1198  AIC: 1202
## log likelihood:  -598.902
## Nagelkerke R2:  0.3123765
## % pres/err predicted correctly:  -281.957
## % of predictable range [ (model-null)/(1-null) ]:  0.2884947
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      4.7096      -0.3971
##
## Degrees of Freedom: 3952 Total (i.e. Null);  3951 Residual
## Null Deviance:      1641
## Residual Deviance: 1494  AIC: 1498
## log likelihood:  -747.2054
## Nagelkerke R2:  0.1074116
## % pres/err predicted correctly:  -379.2816
## % of predictable range [ (model-null)/(1-null) ]:  0.04376851
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr	1172.748	0.0000	1e+00	0.9999995	0.3329613	4.441889	-	-
+ pos							2.301339	0.2255590
preserved ~ CumErr	1201.804	29.0562	5e-07	0.0000005	0.3123765	3.501229	-	NA
							2.587645	
preserved ~ pos	1498.411	325.6632	0e+00	0.0000000	0.1074116	4.709585	NA	-
								0.3970868

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_main_effects_plus_one_model_summary.csv"), row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + I(pos^2) + pos	1172.568	0.0000001	0.0000000	0.9999999	0.3343996	4.48834	-	0.0261061	-	NA	NA
							2.287032		0.4813157		
preserved ~ CumErr + pos	1172.748	0.0000001	0.0000000	0.9999995	0.3329613	4.441889	-	NA	-	NA	NA
							2.301339		0.2255590		
preserved ~ CumErr + CumPres	1185.590	0.0000001	0.0000000	0.9996936	0.3244685	5.024943	-	NA	NA	NA	-
							2.561983				0.1713946
preserved ~ CumErr + stimlen	1196.998	0.0000001	0.0000000	0.9170421	0.3169013	4.569992	-	NA	NA	-	NA
							2.521038			0.1384588	
preserved ~ CumErr	1201.804	29.235420	0.0000000	0.0000000	0.3123765	3.501229	-	NA	NA	NA	NA
							2.587645				
preserved ~ CumErr	1201.804	8056390.090462	50829570	0.3123765	0.3123765	3.501229	-	NA	NA	NA	NA
							2.587645				
preserved ~ CumErr	1201.804	6.213654	0.0003015	0.0003014	0.3123765	3.501229	-	NA	NA	NA	NA
							2.587645				
preserved ~ CumErr	1201.804	29.056196	0.0000000	0.5000000	0.3123765	3.501229	-	NA	NA	NA	NA
							2.587645				
preserved ~ pos	1498.411	325.663176	0.0000000	0.0000000	0.1074116	4.709585	NA	NA	-	NA	NA
									0.3970868		
preserved ~ I(pos^2) + pos	1499.253	26.682378	0.0000000	0.0000000	0.1082436	3.6066732	NA	0.0161532	-	NA	NA
									0.5636646		
preserved ~ stimlen	1606.579	909.580390	0.0000000	0.0000000	0.0287536	0.084536	NA	NA	NA	-	NA
										0.2785772	
preserved ~ CumPres	1630.354	44.763509	0.0000000	0.0000000	0.0111743	2.60193	NA	NA	NA	NA	-
											0.1295356

```
# explore influence of frequency and length

if(grepl("stimlen", BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq")
  )
}
```

```

}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

```

```

## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)         pos      log_freq
##      4.9638      -2.2307      0.0289      -0.4891      0.1599
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3948 Residual
## Null Deviance:      1641
## Residual Deviance: 1152 AIC: 1162
## log likelihood: -576.1649
## Nagelkerke R2: 0.342463
## % pres/err predicted correctly: -278.8528
## % of predictable range [ (model-null)/(1-null) ]: 0.2963003
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)         pos      stimlen      log_freq
##      4.76854      -2.23067      0.02763      -0.48633      0.02757      0.16414
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3947 Residual
## Null Deviance:      1641
## Residual Deviance: 1152 AIC: 1164
## log likelihood: -576.0768
## Nagelkerke R2: 0.3425788
## % pres/err predicted correctly: -278.7834
## % of predictable range [ (model-null)/(1-null) ]: 0.2964748
## *****
## model index: 1

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)      pos
##      4.94883      -2.28703      0.02611      -0.48132
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3949 Residual
## Null Deviance:      1641
## Residual Deviance: 1165 AIC: 1173
## log likelihood: -582.2842
## Nagelkerke R2: 0.3343996
## % pres/err predicted correctly: -278.7011
## % of predictable range [ (model-null)/(1-null) ]: 0.2966818
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)      pos      stimlen
##      5.09721      -2.28601      0.02714      -0.48334      -0.02109
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3948 Residual
## Null Deviance:      1641
## Residual Deviance: 1164 AIC: 1174
## log likelihood: -582.2296
## Nagelkerke R2: 0.3344717
## % pres/err predicted correctly: -278.7505
## % of predictable range [ (model-null)/(1-null) ]: 0.2965576
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.881
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3952 Residual
## Null Deviance:      1641
## Residual Deviance: 1641 AIC: 1643
## log likelihood: -820.6962
## Nagelkerke R2: -6.534383e-16
## % pres/err predicted correctly: -396.6878
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****

```

```

BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

```

```

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	log_freq	stimlen
preserved ~ CumErr + I(pos^2) + pos + log_freq	1162.330	0.000000	1.000000	0.092083	0.424634	0.63771	-	0.0288993	-	0.1599215	NA
							2.230726		0.4891026		
preserved ~ CumErr + I(pos^2) + pos + stimlen + log_freq	1164.154	0.823998	0.401720	0.328490	0.334257	0.8768538	-	0.0276252	-	0.1641400	0.0275724
							2.230667		0.4863266		
preserved ~ CumErr + I(pos^2) + pos	1172.568	0.238791	0.100597	0.004240	0.334399	0.048834	-	0.0261061	-	NA	NA
							2.287032		0.4813157		
preserved ~ CumErr + I(pos^2) + pos + stimlen	1174.452	0.129586	0.002328	0.001647	0.334471	0.097209	-	0.0271366	-	NA	-
							2.286010		0.4833446		0.0210923
preserved ~ 1	1643.392	0.106270	0.000000	0.000000	0.000000	0.000000	0.880535	NA	NA	NA	NA

```

# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

```

```

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

```

```

## Single term deletions
##
## Model:
## preserved ~ CumErr + I(pos^2) + pos + log_freq
##      Df Deviance   AIC
## CumErr    1   1469.8 1477.8
## log_freq    1   1164.6 1172.6
## pos         1   1160.1 1168.1
## I(pos^2)    1   1155.0 1163.0
## <none>       1   1152.3 1162.3

```

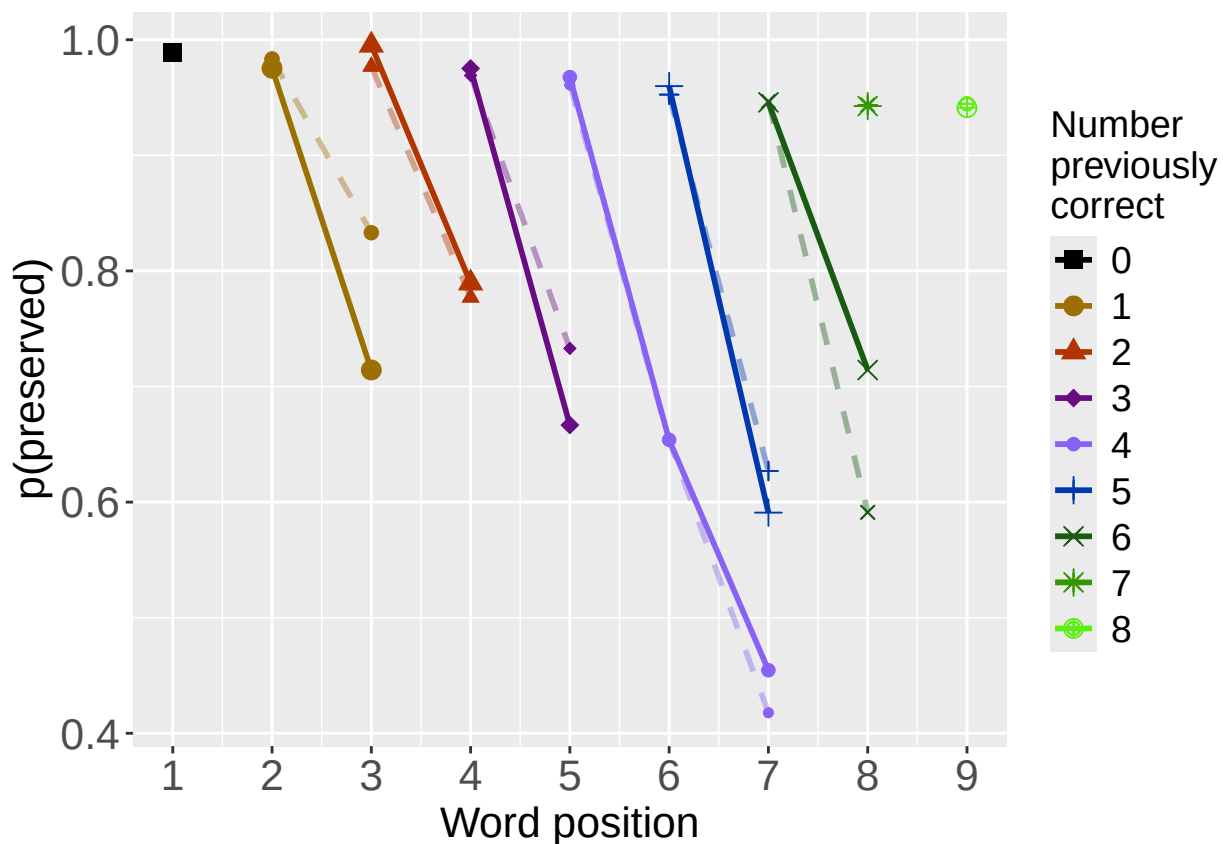
```
#####
# Single deletions from best model
#####

write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                   RunLabel,"_",CurTask,
                                   "_best_model_single_term_deletions.csv"),
          row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

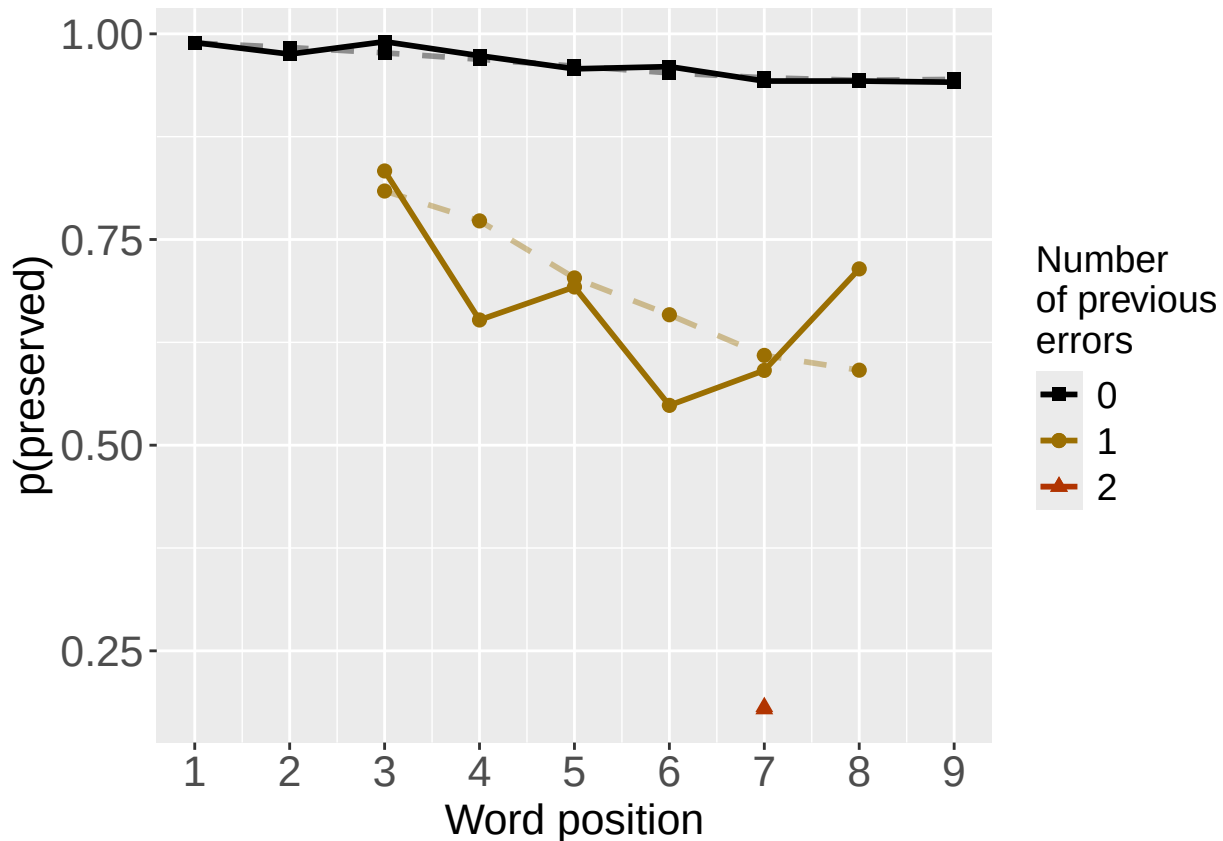
PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",
                 RunLabel,"_",CurTask,
                 "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##          3.501      -2.588
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3951 Residual
## Null Deviance: 1641
## Residual Deviance: 1198 AIC: 1202
## log likelihood: -598.902
## Nagelkerke R2: 0.3123765
## % pres/err predicted correctly: -281.957
## % of predictable range [ (model-null)/(1-null) ]: 0.2884947

```



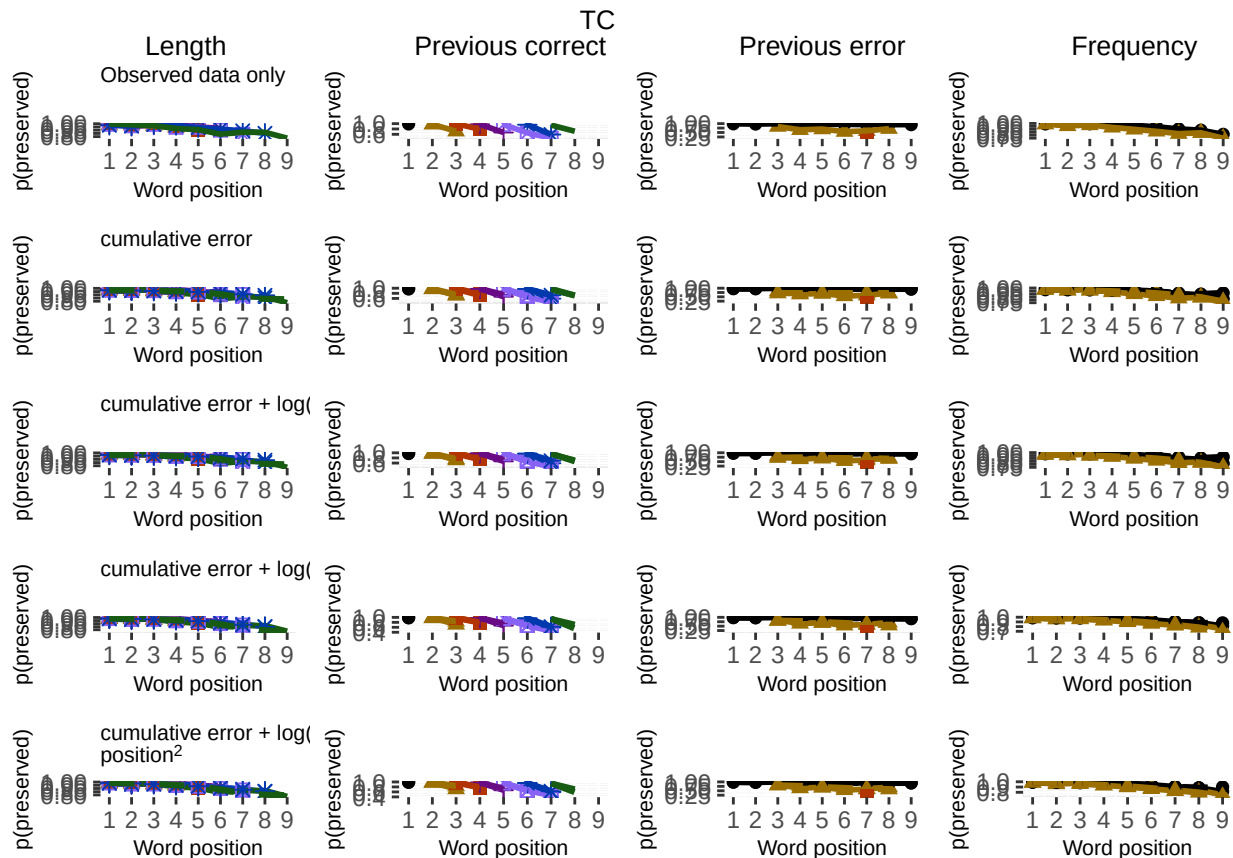
```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      log_freq
##      3.5567      -2.5036      0.1854
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3950 Residual
## Null Deviance:      1641
## Residual Deviance: 1181 AIC: 1187
## log likelihood: -590.3601
## Nagelkerke R2: 0.32372
## % pres/err predicted correctly: -281.8684
## % of predictable range [ (model-null)/(1-null) ]: 0.2887174
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      log_freq      pos
##      4.4086      -2.2460      0.1569     -0.2076
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3949 Residual
## Null Deviance:      1641
## Residual Deviance: 1155 AIC: 1163
## log likelihood: -577.4794
## Nagelkerke R2: 0.340733
## % pres/err predicted correctly: -278.6082
## % of predictable range [ (model-null)/(1-null) ]: 0.2969155
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      log_freq      pos      I(pos^2)
##      4.9638      -2.2307      0.1599     -0.4891      0.0289
##
## Degrees of Freedom: 3952 Total (i.e. Null); 3948 Residual
## Null Deviance:      1641
## Residual Deviance: 1152 AIC: 1162
## log likelihood: -576.1649
## Nagelkerke R2: 0.342463
## % pres/err predicted correctly: -278.8528
## % of predictable range [ (model-null)/(1-null) ]: 0.2963003
## *****
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```



```
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes c
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes c
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```



```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
CurTask,"_dominance_analysis_table.csv"),
row.names = TRUE)
kable(DAContributionAverage)
```

	CumErr	I(pos ²)	pos	log_freq
McFadden	0.2201637	0.0296086	0.0349541	0.0132296

	CumErr	I(pos ²)	pos	log_freq
SquaredCorrelation	0.0851367	0.0119546	0.0140413	0.0052397
Nagelkerke	0.0851367	0.0119546	0.0140413	0.0052397
Estrella	0.1024722	0.0128614	0.0153179	0.0059599

	deviance	deviance_explained
CumErr + log_freq + pos + I(pos^2)	1152.330	489.0627
CumErr + log_freq + pos	1154.959	486.4337
CumErr + log_freq	1180.720	460.6722
CumErr	1197.804	443.5885
null	1641.392	0.0000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                model deviance deviance_explained percent_explained
## CumErr + log_freq + pos + I(pos^2) CumErr + log_freq + pos + I(pos^2) 1152.330      489.0627      29.79560
## CumErr + log_freq + pos              CumErr + log_freq + pos 1154.959      486.4337      29.63543
## CumErr + log_freq                    CumErr + log_freq 1180.720      460.6722      28.06594
## CumErr                              CumErr 1197.804      443.5885      27.02513
## null                                null 1641.392        0.0000      0.00000
##                                percent_of_explained_deviance increment_in_explained
## CumErr + log_freq + pos + I(pos^2) 100.00000      0.5375592
## CumErr + log_freq + pos              99.46244      5.2675308
## CumErr + log_freq                    94.19491      3.4931474
## CumErr                              90.70176      90.7017626
## null                                NA              0.0000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
```

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + log_freq + pos + I(pos ²)	29.79560	100.00000	0.5375592
CumErr + log_freq + pos	29.63543	99.46244	5.2675308
CumErr + log_freq	28.06594	94.19491	3.4931474
CumErr	27.02513	90.70176	90.7017626
null	0.00000	NA	0.0000000

```

                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##           Nagelkerke
## CumErr    0.73158947
## I(pos^2)  0.10272727
## pos       0.12065834
## log_freq  0.04502492

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

```

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_sse_results_table.csv"),row.names = TRUE)

sse_table

##               model p_accounted_for model_deviance diff_CumErr+log_freq+pos diff_CumErr diff_CumErr+log_freq
## 1      preserved ~ CumErr+log_freq+pos      0.9574755      1154.959      0.000000000 -0.008397482      -0.009515898
## 2      preserved ~ CumErr      0.9658729      1197.804      0.008397482      0.000000000      -0.001118416
## 3      preserved ~ CumErr+log_freq      0.9669914      1180.720      0.009515898      0.001118416      0.000000000
## 4 preserved ~ CumErr+log_freq+pos+I(pos^2)      0.9681375      1152.330      0.010662083      0.002264601      0.001146185
## diff_CumErr+log_freq+pos+I(pos^2)
## 1      -0.010662083
## 2      -0.002264601
## 3      -0.001146185
## 4      0.000000000

```

model	p_accounted_for	model_deviance
preserved ~ CumErr+log_freq+pos	0.9574755	1154.959
preserved ~ CumErr	0.9658729	1197.804
preserved ~ CumErr+log_freq	0.9669914	1180.720
preserved ~ CumErr+log_freq+pos+I(pos^2)	0.9681375	1152.330

model	diff_CumErr+log_freq+pos	diff_CumErr	diff_CumErr+log_freq	diff_CumErr+log_freq+pos+I(pos^2)
preserved ~ CumErr+log_freq+pos	0.0000000	-0.0083975	-0.0095159	-0.0106621
preserved ~ CumErr	0.0083975	0.0000000	-0.0011184	-0.0022646
preserved ~ CumErr+log_freq	0.0095159	0.0011184	0.0000000	-0.0011462
preserved ~ CumErr+log_freq+pos+I(pos^2)	0.0106621	0.0022646	0.0011462	0.0000000

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF, paste0(TablesDir, CurPat, "_", RunLabel, "_",
  CurTask, "_results_report_df.csv"), row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```